FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

SEM II – 2024/2025

PROJECT 1 - CLASSIFICATION

DATA MINING - SECP2753-02

NAME	MATRIC NO
MUHAMMAD AFIQ DANIAL BIN ROZAIDIE	A23CS0117
MOHAMED ALIF FATHI BIN ABDUL LATIF	A23CS0112

LECTURER:

ROZILAWATI BINTI DOLLAH @ MD. ZAIN

Table of Contents

1.0 Introduction

Heart disease is one of the most common and serious health issues in today's world. This disease still remains a significant threat and a burden towards the healthcare system and also affecting several individuals each year even today. The early detection to prevent heart related conditions can be crucial to improving patients outcome and reducing healthcare costs. Factors such as smoking, gender, age, blood pressure, cholesterol levels, and a family history of heart disease play a significant role in its development.

Given the nature of heart diseases as it is, traditional diagnostic methods do not always capture the complexity of individual risk profiles. Meanwhile, machine learning can provide a promising approach by using the characteristics of each dataset to uncover the hidden patterns, analyze interactions between variables to make accurate predictions.

This project aims to analyze these risk factors and utilize machine learning techniques to develop a predictive model for heart disease. The supervised machine learning algorithms that will be used in this project are Naive Bayes, XGBoost and K-Nearest Neighbors. By applying all of these algorithms, the objective to develop a classification model is to be able to predict a certain class from each business insight that has been stated. This process will involve many key stages including data preprocessing, model training, validation, and evaluation using appropriate performance metrics.

Ultimately, this machine learning seeks to contribute to early diagnosis and preventive healthcare measures. It also can be a tool to assist healthcare professionals in making accurate decisions and inform individuals to take precautions in managing their own health.

2.0 Business insight

There are several business insights that were determined from assignment 1. There are only a few that are chosen as the most relatable for this project.

2.1 Heart Disease Status Detection

This insight will show the early detection of heart disease based on all attributes in a dataset. Correlation analysis further highlights critical risk factors like age, BMI, and diabetes status, supporting more accurate risk prediction models.

2.2 Family History Influence

This analysis evaluates how a family history of heart disease affects an individual's own risk. By comparing the rate of heart disease between those with and without a recorded family history, we can determine the increase in risk. A statistical analysis can be used to confirm whether this difference is significant, thereby supporting the accuracy of the analysis, which shows that family history is a factor in heart disease.

2.3 Impact of Diabetes

This insight will solely focus on the relationship between diabetes and heart disease. By analyzing their diabetes status, we can observe how the influence of diabetes correlates with higher instances of heart disease. This analysis will highlight that the individual with diabetes will have a significantly elevated risk of getting heart conditions compared to non-diabetic individuals.

3.0 Data Preprocessing and Sampling

Data Preprocessing and Sampling are essential steps in preparing data for machine learning models. Data Preprocessing involves cleaning and transforming raw data into a format that can be effectively used by all three classification algorithms. While the sampling refers to the technique of selecting a subset of data from a larger dataset evenly.

3.1 Data Cleaning

To prepare the dataset for classification, several preprocessing steps were performed. Missing values were handled by filling in numerical attributes with their mean and categorical attributes with their mode. For instance, attributes such as Cholesterol Level and BMI were filled with the mean, while Gender was filled with the mode.

Duplicates were checked using the df.duplicated().sum() function, which returned zero. This indicate no duplicate entries were found or removed.

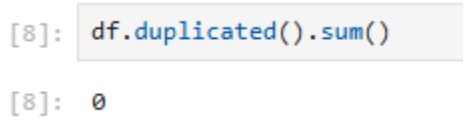


Figure 3.1: Total Duplicated in Dataset

Next, columns with a high percentage of missing values were dropped because high levels of missingness can compromise the accuracy in prediction of the dataset. This resulted in the removal of the Alcohol Consumption attribute due to having approximately 25% missing values.

```
[29]: missing_percent = df.isnull().mean() * 100
      print(missing_percent)

Age                       0.29
Gender                    0.19
Blood Pressure            0.19
Cholesterol Level         0.30
Exercise Habits           0.25
Smoking                   0.25
Family Heart Disease      0.21
Diabetes                  0.30
BMI                       0.22
High Blood Pressure       0.26
Low HDL Cholesterol       0.25
High LDL Cholesterol      0.26
Alcohol Consumption      25.86
Stress Level              0.22
Sleep Hours               0.25
Sugar Consumption         0.30
Triglyceride Level        0.26
Fasting Blood Sugar       0.22
CRP Level                 0.26
Homocysteine Level        0.20
Heart Disease Status      0.00
dtype: float64
```

Figure 3.2: Missing Percentage for Every Attributes

3.2 Encoding Categorical Variables

Categorical attributes were transformed into numerical values to make them suitable for classification models. There are two types of encoding methods were applied:

- Binary Label Encoding was used for attributes with two categories, such as Gender, Smoking, Family Heart Disease, Diabetes, High Blood Pressure, Low HDL Cholesterol, High LDL Cholesterol, and Heart Disease Status. These values were encoded using LabelEncoder(), mapping each category to either 0 or 1.

- Ordinal Mapping was applied to attributes with ordered categories like Exercise Habits, Stress Level, and Sugar Consumption. These were mapped to numerical values based on their order which are Low = 0, Medium = 1, and High = 2. This method will maintain the ordinal relationship between the values, which is important for algorithms that can utilize ranking information.

3.3 Sampling

After data cleaning, the dataset was split into training and testing sets using an 80:20 ratio. To maintain consistent class distribution between the two sets, stratified sampling was used, which is really important in classification tasks to avoid biased performance evaluation.

```
target_column = 'Heart Disease Status'

# Split into features and target
x = df.drop(target_column, axis=1)
y = df[target_column]

# Train/Test split|
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, stratify = y, random_state=42)
```

Figure 3.3: Splitting Training and Testing Code in Python

3.4 Feature Scaling

Since the K-Nearest Neighbors (KNN) algorithm is sensitive to the scale of attributes due to its reliance on distance calculations, standardization was applied using StandardScaler. This method transforms the attributes to have a mean of 0 and a standard deviation of 1. Standardization ensures that all numerical attributes contribute equally to the distance metric used in KNN. This method will enhance model accuracy and prevent attributes with larger scales from influencing the learning process.

```
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

Figure 3.4: StandardScaler Code in Python

4.0 Classification Algorithm

For a numerous types of algorithm for classification, we only use Naive Bayes, XGBoost and K-Nearest Neighbors as our choice of algorithm in this project. Each of these models will be used in the Python programming language throughout this project.

4.1 Naive Bayes

Naive Bayes is a classification algorithm that uses probability to predict which category a data point belongs to, assuming that all features are unrelated. It is named as "Naive" because it assumes the presence of one feature does not affect other features. The "Bayes" part of the name refers to its basis in Bayes' Theorem [3]. Naive Bayes is one of the simplest yet surprisingly powerful classification algorithms based on Bayes' Theorem. Despite its simplicity, it performs remarkably well in many real-world situations. GaussianNB classifier imported from scikit-learn, a machine learning library in Python is used to train, predict and evaluate the training data . This figure below will show the partition code of using Naive Bayes model.

```python
from sklearn.naive_bayes import GaussianNB

#Train Naive Bayes model
model = GaussianNB()
model.fit(x_train, y_train)

#Predict and evaluate
y_pred = model.predict(x_test)
cm = confusion_matrix(y_test, y_pred)
```

Figure 4.1*: *Naive Bayes Classification Model in Python

4.2 XGBoost

XGBoost (Extreme Gradient Boosting) algorithm is a highly efficient and scalable machine learning model well-suited for classification tasks. It is a powerful implementation of gradient boosting that often outperforms other models, especially when capturing complex patterns in data [2]. The reason XGBoost is chosen is because it can capture both linear and nonlinear patterns in the data using its tree-based architecture. This model also can handle imbalanced data with class weights. This flexibility is especially useful in this case, where the target class is imbalanced (Yes: 20%, No: 80%). XGBClassifier from the Xgboost Python library used to train and evaluate the classification model. The figure below shows the code to use the XGBoost model using python.

```python
from xgboost import XGBClassifier

# Train XGBoost model
model = XGBClassifier(random_state=42)
model.fit(x_train, y_train)

# Predict and evaluate
y_pred = model.predict(x_test)
cm = confusion_matrix(y_test, y_pred)
```

Figure 4.2: XGBoost classification Model Code in Python

4.3 K-NN

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm generally used for classification but can also be used for regression tasks. It works by finding the "k" closest data points (neighbors) to a given input and makes predictions based on the majority class (for classification) or the average value (for regression). Since KNN makes no assumptions about the underlying data distribution it makes it a non-parametric and instance-based learning method [4]. The value of k in KNN will be 10 to balance between overfitting and underfitting, leading to higher classification accuracy on new data. KNeighborsClassifier was imported in order to use a KNN model. StandardScaler was also important to be imported in order to use a Euclidean distance to find the nearest neighbors. This figure below shows the part of code in Python using a KNN classifier.

```python
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

#Normalize data (IMPORTANT for KNN)
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)

#Train KNN model
knn = KNeighborsClassifier(n_neighbors=10)
knn.fit(x_train_scaled, y_train)

#Predict
y_pred = knn.predict(x_test_scaled)

#Evaluate with Heatmap Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
```

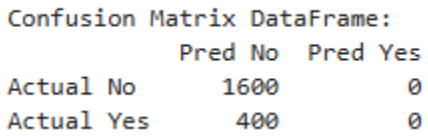
Figure 4.3*: *K-Nearest Neighbors Classification Model in Python

5.0 Discussion

The discussion focuses on explaining the evaluation metrics which are accuracy, precision, recall, and F1-score. These metrics assess the effectiveness of each model in predicting target variables. These metrics provide a more comprehensive view of the model's strengths and weaknesses. The comparison will show us the meaningful insights regarding each algorithm's predictive capabilities and suitability for this health disease classification task.

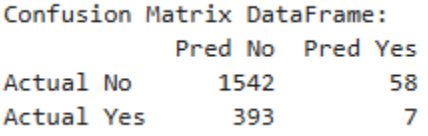
5.1 Insight 1: Heart Disease Status Detection

For this insight, attribute "Heart Disease Status" chosen to be the target variable. To further illustrate how each model performed in terms of classifying heart disease cases, a confusion matrix heatmap is presented for each algorithm.



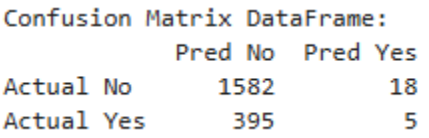
```
Confusion Matrix DataFrame:
            Pred No  Pred Yes
Actual No      1600         0
Actual Yes      400         0
```

Figure 5.1: Confusion Matrix Heatmap for Naive Bayes in Heart Disease Status Detection Insight



```
Confusion Matrix DataFrame:
            Pred No  Pred Yes
Actual No      1542        58
Actual Yes      393         7
```

Figure 5.2: Confusion Matrix Heatmap for XGBoost in Heart Disease Status Detection Insight



```
Confusion Matrix DataFrame:
            Pred No  Pred Yes
Actual No      1582        18
Actual Yes      395         5
```

Figure 5.3: Confusion Matrix Heatmap for KNN in Heart Disease Status Detection Insight

The table below shows the performance metric for accuracy, precision, recall and F1-Score on Heart Disease Status Detection Insight.

Algorithm/ result(%)	Accuracy	Precision	Recall	F1-Score
Naive Bayes	80.00	64.00	80.00	71.11
XGBoost	77.45	65.91	77.45	70.40
K-Nearest Neighbors	79.35	68.36	79.35	71.24

Table 5.1: Comparison Between Naive Bayes, XGBoost and K-Nearest Neighbors for Heart Disease Status Detection Insight

Based on the comparison table, it shows that Naive Bayes achieved the highest accuracy (80.00%). However, its precision (64.00%) was the lowest among the three, indicating a higher number of false positives. This is also reflected in the heatmap, where the model failed to predict the 'Yes' class entirely, resulting in zero true positives. KNN delivered the highest F1-score (71.24%), showing a good balance between precision and recall, and had a competitive accuracy of 79.35%. Meanwhile, XGBoost performed consistently across all metrics, with a good F1-score of 70.40%, slightly lower than KNN but with better precision than Naive Bayes. Overall, KNN and XGBoost provided more balanced predictions.

5.2 Insight 2: Family History Influence

For this insight, attribute “Family Heart Disease” chosen to be the target variable. To further illustrate how each model performed in terms of classifying heart disease cases, a confusion matrix heatmap is presented for each algorithm.

Confusion Matrix DataFrame:

	Pred No	Pred Yes
Actual No	543	462
Actual Yes	505	490

Figure 5.4: Confusion Matrix Heatmap for Naive Bayes in Family History Influence Insight

Confusion Matrix DataFrame:

	Pred No	Pred Yes
Actual No	530	475
Actual Yes	510	485

Figure 5.5: Confusion Matrix Heatmap for XGBoost in Family History Influence Insight

Confusion Matrix DataFrame:

	Pred No	Pred Yes
Actual No	646	359
Actual Yes	619	376

Figure 5.6: Confusion Matrix Heatmap for KNN in Family History Influence Insight

The table below shows the performance metric for accuracy, precision, recall and F1-Score on Family History Influence Insight.

Algorithm/ result(%)	Accuracy	Precision	Recall	F1-Score
Naive Bayes	51.65	51.64	51.65	51.62
XGBoost	50.75	50.74	50.75	50.73
K-Nearest Neighbors	51.1	51.11	51.1	50.23

Table 5.2: Comparison Between Naive Bayes, XGBoost and K-Nearest Neighbors for Family History Influence Insight

This result shows that all three models performed poorly predicting the "Family Heart Disease" attribute, with accuracy, precision, recall, and F1-scores hovering around 50%. This indicates that the models struggled to find meaningful patterns related to family history from the available attributes. The confusion matrix heatmaps also support this, showing nearly balanced misclassifications between the classes, which further suggests limited predictive power for this particular target variable. The possibilities of the low performance because it lacks strong predictive signals related to family heart disease history. The target variable not being well correlated with this attribute may also be a reason for this poor result.

5.3 Insight 3: Impact of Diabetes

For this insight, attribute “Diabetes” chosen to be the target variable. To further illustrate how each model performed in terms of classifying heart disease cases, a confusion matrix heatmap is presented for each algorithm.

Confusion Matrix DataFrame:

	Pred No	Pred Yes
Actual No	577	433
Actual Yes	563	427

Table 5.7: Confusion Matrix for Naive Bayes Classifier in Impact of Diabetes

Confusion Matrix DataFrame:

	Pred No	Pred Yes
Actual No	493	517
Actual Yes	506	484

Table 5.8: Confusion Matrix for XGBoost Classifier in Impact of Diabetes

Confusion Matrix DataFrame:

	Pred No	Pred Yes
Actual No	649	361
Actual Yes	624	366

Table 5.9: Confusion Matrix for KNN Classifier in Impact of Diabetes

The table below shows the performance metric for accuracy, precision, recall and F1-Score on Diabetes insight.

Algorithm/ result	Accuracy	Precision	Recall	F1-Score
Naive Bayes	50.2	50.14	50.2	49.96
XGBoost	48.85	48.86	48.85	48.85
K-Nearest Neighbors	50.75	50.67	50.75	49.81

Table 5.3: Comparison Between Naive Bayes, XGBoost and K-Nearest Neighbors for Impact of Diabetes Insight

Based on the overall performance on those three algorithms on Naive Bayes, XGBoost, and K-Nearest Neighbors which is relatively poor, with accuracy, precision, recall, and F1-scores all hovering around the 50% mark. This level of performance is only slightly better than random guessing, especially if the classification task is binary or balanced, indicating that none of the models are effectively capturing meaningful patterns for indicating diabetes in the data. Such low scores suggest potential issues with the dataset, such as poor feature quality, class imbalance, noise, or insufficient preprocessing. To extract more valuable insights about diabetes and its impact, further data refinement, feature engineering, and possibly the inclusion of more informative health variables would be necessary to improve model reliability and decision-making support.

6.0 Conclusion

This project successfully applied three supervised classification algorithms which are Naive Bayes, XGBoost, and K-Nearest Neighbors. These models analyze heart disease risk factors and develop predictive models. The heart disease status detection proved to be the most successful insight, with Naive Bayes achieving the highest accuracy of 80.00% and K-Nearest Neighbors providing the most balanced performance with better F1-score of 71.24%. However, the family history influence and impact of diabetes classifications have poor results, with all models performing at approximately 50% accuracy. This shows that the attributes used in training may lack sufficient predictive signals for these target variables.

The variation in model performance across different target variables highlights the importance of specific algorithm selection in heart disease prediction tasks. The successful heart disease status detection model shows the potential for machine learning to assist healthcare professionals in diagnosis and analysis of the risk of heart disease. The poor result in predicting family history and diabetes shows the complexity of analyzing healthcare data.

To improve model performance, future work should consider other methods for handling the dataset's class imbalance (20% Yes, 80% No for heart disease) and handling low correlation dataset. This project taught us the challenges of applying machine learning for healthcare prediction tasks to provide valuable insights for future research in medical data mining and predictive healthcare analytics systems.

7.0 References

[1] Know your risk: Family history and heart disease | Heart Foundation. (n.d.). https://www.heartfoundation.org.au/your-heart/family-history-and-heart-disease

[2] GeeksforGeeks. (2025, May 23). *XGBoost*. GeeksforGeeks. https://www.geeksforgeeks.org/xgboost/

[3] GeeksforGeeks. (2025, May 21). *Naive Bayes classifiers*. GeeksforGeeks. https://www.geeksforgeeks.org/naive-bayes-classifiers/

[4] GeeksforGeeks. (2025a, May 14). *KNeArest Neighbor(KNN) algorithm*. GeeksforGeeks. https://www.geeksforgeeks.org/k-nearest-neighbours/

[5] Heart disease. (2024, December 29). Kaggle. https://www.kaggle.com/datasets/oktayrdeki/heart-disease